

Templates : Template/PG-fr.docx | Template/Presentation_BELACEL_V3.pptx

PDF : TP_Informatique_ENS_1ère_année_AN_07.pdf

Instructor: BELACEL Madani

Module: Computer Science

Level: ENS — Year 1 — English Department

Duration: 1h30

Academic Year: 2025-2026

Learning Objectives

At the end of this lab, the student is able to:

- Read and write text files in Python
- Manipulate the CSV format (reading, writing, analysis)
- Use the csv module in Python
- Process real data (student grades, results)
- Generate automated reports
- Create reusable teaching tools (revision cards, logbook)

Lab Plan

Phase	Duration	Activity
Welcome and review	5 min	Recap on files
Exercise 1	15 min	Reading and writing text files
Exercise 2	20 min	Creating a CSV grades file
Exercise 3	20 min	CSV data analysis
Exercise 4	20 min	Revision card generator
Exercise 5	10 min	Session logbook

Exercise 1 — Reading and Writing Text Files (15 min)

Problem statement: Create a script `journal.py` that:

1. Asks the user to enter 3 sentences (their daily journal)
2. Saves each sentence to a file `journal.txt` (one sentence per line)
3. Reads the file and displays its numbered content

Expected example:

```
Sentence 1: Today I learned about files in Python.  
Sentence 2: It's useful for saving data.  
Sentence 3: I'm going to create a quiz project.
```

```
=== My Journal ===
```

1. Today I learned about files in Python.
2. It's useful for saving data.
3. I'm going to create a quiz project.

Solution:

```
with open("journal.txt", "w", encoding="utf-8") as f:
    for i in range(3):
        phrase = input(f"Sentence {i+1}: ")
        f.write(phrase + "\n")

print("\n=== My Journal ===")
with open("journal.txt", "r", encoding="utf-8") as f:
    for i, line in enumerate(f, 1):
        print(f"{i}. {line.strip()}")
```

Exercise 2 — Creating a CSV Grades File (20 min)

Problem statement: Create a script `creer_notes.py` that:

1. Asks for the number of students
2. For each student, asks for name, grade1, grade2, grade3
3. Saves to `notes.csv` with a header
4. Displays a summary of the recorded data

```
flowchart LR
    A[User input] --> B[notes.csv]
    B --> C[Read by analyse_notes.py]
    C --> D[Calculate averages]
    D --> E[resultats.csv]
    style B fill:#f9f,stroke:#333,stroke-width:2px
    style E fill:#9cf,stroke:#333,stroke-width:2px
```

Solution:

```
import csv

nb = int(input("Number of students: "))
students = []

for i in range(nb):
    print(f"\n--- Student {i+1} ---")
    name = input("Name: ")
    g1 = float(input("Grade 1: "))
    g2 = float(input("Grade 2: "))
    g3 = float(input("Grade 3: "))
    students.append([name, g1, g2, g3])

with open("notes.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.writer(f)
    writer.writerow(["Name", "Grade1", "Grade2", "Grade3"])
    writer.writerows(students)

print(f"\n✓ {nb} students saved in notes.csv")
```

Exercise 3 — CSV Data Analysis (20 min)

Problem statement: Create a script `analyse_notes.py` that reads `notes.csv` and calculates:

1. Each student's average
2. The class average
3. The number of passing students (average ≥ 10)
4. Exports the results to `resultats.csv`

Expected example:

```
=== Grade Analysis ===
Ali → Average: 14.00 – ✓ Passed
Sara → Average: 16.67 – ✓ Passed
Mohamed → Average: 9.00 – ✗ Failed
Leila → Average: 16.00 – ✓ Passed
Ahmed → Average: 9.33 – ✗ Failed

Class average: 13.00
Students passed: 3/5
Results exported to resultats.csv
```

Solution:

```
import csv

try:
    with open("notes.csv", "r", encoding="utf-8") as f:
        pass
except FileNotFoundError:
    with open("notes.csv", "w", newline="", encoding="utf-8") as f:
        writer = csv.writer(f)
        writer.writerow(["Name", "Grade1", "Grade2", "Grade3"])
        writer.writerow(["Ali", 14, 12, 16])
        writer.writerow(["Sara", 18, 15, 17])
        writer.writerow(["Mohamed", 8, 10, 9])
        writer.writerow(["Leila", 16, 14, 18])
        writer.writerow(["Ahmed", 10, 11, 7])

students = []
with open("notes.csv", "r", encoding="utf-8") as f:
    reader = csv.DictReader(f)
    for row in reader:
        grades = [float(row["Grade1"]), float(row["Grade2"]), float(row["Grade3"])]
        average = sum(grades) / len(grades)
        students.append({"name": row["Name"], "grades": grades, "average": average})

with open("resultats.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.writer(f)
    writer.writerow(["Name", "Grade1", "Grade2", "Grade3", "Average", "Passed"])
    for s in students:
        passed = "Yes" if s["average"] >= 10 else "No"
        writer.writerow([s["name"], s["grades"][0], s["grades"][1],
            s["grades"][2], f"{s['average']:.2f}", passed])

averages = [s["average"] for s in students]
class_average = sum(averages) / len(averages)
passed_count = sum(1 for m in averages if m >= 10)

print("=== Grade Analysis ===")
for s in students:
    status = "✓ Passed" if s["average"] >= 10 else "✗ Failed"
    print(f"{s['name']}<10} → Average: {s['average']:.2f} – {status}")
```

```
print(f"\nClass average: {class_average:.2f}")
print(f"Students passed: {passed_count}/{len(students)}")
print("Results exported to resultats.csv")
```

Exercise 4 — Revision Card Generator (20 min)

Problem statement: Create a script `fiches_revision.py` that allows creating and using revision cards. The program:

1. Loads cards from a CSV (concept, definition)
2. Starts a quiz: displays the concept, user gives the definition
3. Marks as correct or not
4. Displays the final score

```
flowchart TD
    A[Start] --> B[Create / load CSV cards]
    B --> C[Shuffle cards]
    C --> D[Display concept]
    D --> E[User enters definition]
    E --> F[Display expected definition]
    F --> G[Self-assessment: Correct?]
    G --> H[More cards?]
    H -->|Yes| D
    H -->|No| I[Display final score]
    I --> J[End]
    style B fill:#f9f,stroke:#333,stroke-width:2px
```

Solution:

```
import csv
import random

def create_cards(filename):
    """Creates a CSV file of revision cards."""
    concepts = [
        ("Variable", "A box that stores a value"),
        ("Function", "A reusable block of code that performs a task"),
        ("Loop", "A structure that repeats an instruction"),
        ("Condition", "A structure that executes based on whether a condition is true"),
        ("Array", "A collection of elements accessible by index"),
        ("Stack", "LIFO structure – last in, first out"),
        ("Queue", "FIFO structure – first in, first out"),
        ("Algorithm", "A finite sequence of instructions to solve a problem")
    ]
    with open(filename, "w", newline="", encoding="utf-8") as f:
        writer = csv.writer(f)
        writer.writerow(["Concept", "Definition"])
        writer.writerows(concepts)
    print(f"{len(concepts)} cards created in {filename}")

def load_cards(filename):
    """Loads cards from a CSV file."""
    cards = []
    with open(filename, "r", encoding="utf-8") as f:
        reader = csv.DictReader(f)
        for row in reader:
            cards.append((row["Concept"], row["Definition"]))
    return cards

def run_quiz(cards):
```

```

"""Runs a revision quiz."""
random.shuffle(cards)
score = 0
print("\n=== Revision Quiz ===\n")
for concept, definition in cards:
    print(f"Concept: {concept}")
    answer = input("Your definition: ")
    print(f"Expected definition: {definition}")
    correct = input("Correct? (y/n): ").lower()
    if correct == "y":
        score += 1
    print()
print(f"Final score: {score}/{len(cards)}")

filename = "fiches_revision.csv"
create_cards(filename)
cards = load_cards(filename)
run_quiz(cards)

```

Exercise 5 — Session Logbook (10 min)

Problem statement: Create a script `journal_bord.py` that records a timestamped work logbook. The program:

1. Asks for the student's name
2. Asks for subjects studied and time spent
3. Saves to a CSV with automatic timestamp
4. Displays a session summary

Expected example:

```

Student name: Sara
Subject studied (or 'done'): Maths
Time spent on 'Maths' (min): 45
Subject studied (or 'done'): Python
Time spent on 'Python' (min): 60
Subject studied (or 'done'): done

=== Session summary for 2026-06-25 ===
Student: Sara
Number of subjects: 2
• Maths: 45 minutes
• Python: 60 minutes
Total study time: 105 minutes (1.8 hours)
Data saved in journal_bord.csv

```

Solution:

```

import csv
from datetime import date

name = input("Student name: ")
today = date.today()

subjects = []
while True:
    subject = input("Subject studied (or 'done'): ")
    if subject == "done":
        break
    time = int(input(f"Time spent on '{subject}' (min): "))

```

```

subjects.append((subject, time))

with open("journal_bord.csv", "a", newline="", encoding="utf-8") as f:
    writer = csv.writer(f)
    if f.tell() == 0:
        writer.writerow(["Date", "Name", "Subject", "Time_min"])
    for subject, time in subjects:
        writer.writerow([today, name, subject, time])

total_time = sum(t for _, t in subjects)
print(f"\n=== Session summary for {today} ===")
print(f"Student: {name}")
print(f"Number of subjects: {len(subjects)}")
for s, t in subjects:
    print(f" • {s}: {t} minutes")
print(f"Total study time: {total_time} minutes ({total_time/60:.1f} hours)")
print("Data saved in journal_bord.csv")

```

Self-Assessment MCQ

- Which Python function opens a file?
 - open() ✓
 - file()
 - read()
- What does CSV stand for?
 - Comma-Separated Values ✓
 - Computer System Validation
 - Common Source Variables
- Which open mode allows writing at the end of an existing file?
 - "w"
 - "a" ✓
 - "r"
- Which method reads all lines of a CSV file as dictionaries?
 - csv.reader()
 - csv.DictReader() ✓
 - csv.writer()
- Why do we use newline="" with csv.writer()?
 - To avoid blank lines between rows ✓
 - To remove special characters
 - It's not necessary

Grading Rubric — Lab 07

Criterion	Weight	Not acquired (0)	Partial (0.5)
-----------	--------	------------------	---------------

Text file reading/writing	20%	Does not master <code>open()</code>	Basic use
CSV module (reading)	20%	Cannot read a CSV	Reading with e
CSV module (writing)	20%	Cannot write a CSV	Partial writing
Data processing	20%	No processing	Partial process
Result export	20%	No export	Partial export

Session Schedule — 1h30 (90 min)

Phase	Duration	Cumulative total
Welcome	5 min	5 min
Exercise 1 — Text journal	15 min	20 min
Exercise 2 — Creating CSV grades	20 min	40 min
Exercise 3 — CSV analysis	20 min	60 min
Exercise 4 — Revision cards	20 min	80 min
Exercise 5 — Logbook	10 min	90 min
Total	90 min	

Pedagogical Note ENS

Teaching transposition

This lab is the most concrete of the eight sessions: it manipulates real data that speaks to future teachers.

- Exercises 2 and 3 (CSV grades): These exercises simulate a teacher's daily work: entering grades, calculating averages, exporting results. Students realize that a Python script can replace a spreadsheet for repetitive tasks.
- Exercise 4 (revision cards): This is the most directly reusable exercise. Future teachers can adapt this program for any discipline.

Trainer advice

Ensure students understand the CSV format (comma separator, quotes for strings). Show how to open a CSV with a spreadsheet (LibreOffice Calc or Excel) to visualize the result. For middle school students, simplify by providing a pre-filled CSV file and only ask for reading and display.

Extension

Create a complete "Class Grade Management" system that reads a CSV of students, calculates subject averages, generates an individual report card in text format, and exports results to a new CSV with comments.